Trabalho Prático 02 (T2)

Identificao da Disciplina

Cdigo da Disciplina	Nome da Disciplina	Professor	Perodo
CIC_1MA e EGS_1MA	Algoritmos e Lógica de Programação	Jeremias Moreira Gomes	2023/1

1. Objetivo

O objetivo deste trabalho é desenvolver um jogo 2D, por meio do terminal.

Ao final do T2, o aluno deverá ter: (*i*) exercitado suas habilidades de programação contemplando estruturas condicionais, de repetição, funções, estruturas homegêneas e heterogêneas, utilização de matrizes e arquivos; (*ii*) dado o enunciado, ter resolvido um problema, elaborando um algoritmo para solucioná-lo; e (*iii*) desenvolver um labirinto 2D com navegação que funcione corretamente em um computador.

2. Enunciado

Após o sucesso do seu primeiro jogo trabalhando na *Immersive Digital Play* (IDP), você recebeu uma nova tarefa que é desenvolver uma extensão para o seu primeiro jogo. Se o RPG de texto foi um sucesso absoluto, dessa vez eles querem que você implemente um RPG 2D. Nessa nova aventura haverá um personagem fictício, montado pelo jogador, com alguns atributos e que irá percorrer um labirinto com monstros e desafios. O objetivo final do jogo é encontrar o chefe do labirinto e derrotá-lo. Além disso, o jogo poderá ter diferentes níveis de dificuldade para aqueles gamers mais hardcores.

3. Metodologia

O jogo deverá ser desenvolvido em quatro partes: (*i*) Criação de um menu de apresentação; (*ii*) Criação de personagem; (*iii*) Criação de um labirinto; e *(iv)* Criação de eventos.

3.1. Menu de entrada (10 pts)

Crie um breve diálogo mostrando 3 opções ao usuário. Opções:

1 Criar Personagem
2 Iniciar Jogo
0 Sair

O usuário só sairá do jogo, se acessar a opção de número 0 (Sair). Caso contrário, terá acesso a parte de criação de personagem (opção 1), inicialização do jogo (opção 2). A opção de iniciar jogo só poderá ser acessada caso um personagem já tenha sido criado (opção 1). A primeira opção poderá ser acessada quantas vezes o jogador quiser para alterar as características do personagem antes do jogo começar.

3.2. Criação de Personagem (15 pts)

O jogador deverá ser capaz de criar seu personagem com as seguintes características:

- Raça (1 - Humano, 2 - Anão, 3 - Elfo)
- Alinhamento (1 Mal, 2 Neutro e 3 Bom)
- Profissão (1 - Guerreiro, 2 - Mago, 3 - Ladino)
- História prévia - Como o personagem viveu até aquele momento (Texto de 400 caracteres no máximo)
- Porte (1 Grande, 2 Médio, 3 Pequeno)
- Força (Um inteiro de 1 até 5)
- Destreza (Um inteiro de 1 até 5)
- Inteligência (Um inteiro de 1 até 5)
- Constituição (Um inteiro de 1 até 5)

Restrições

1. Quando um personagem for da raça Anão, ele só poderá escolher o porte como Médio ou Pequeno.
2. Quando um personagem for de alinhamento Bom, ele não poderá escolher a classe Ladino.
3. A soma dos atributos de Força + Constituição + Destreza + Inteligência não pode superar o total de 10 pontos, sendo que cada atributo deve ter o valor mínimo de 1 e máximo de 5.
4. Quando o jogador decide pela raça Elfo, ele soma 1 de Destreza e -1 de Constituição.
5. Quando o jogador decide pela raça Anão, ele soma 1 de Força e -1 de Destreza.
6. Quando o jogador decide pela profissão Mago, ele soma 1 de Inteligência e -2 de força.
7. Quando o jogador decide pela profissão Ladino, ele soma 1 de destreza e -1 de força e -1 de Inteligência.
8. Os ataques em jogo deverão ser associados a classe de personagem, por exemplo, o Guerreiro ataca utilizando força, o Mago utiliza Inteligência e o Ladino utiliza a Destreza, sendo seus atributos principais.
9. Os pontos de vida do personagem são calculados a partir de sua constituição. Isto é, o valor incial dos pontos de vida é 20, os pontos de vida do personagem serão dados pela multiplicação entre esse valor e a constituição (`constituição * 20`). Repare que é possível criar um personagem com a constituição igual zero, mas somente através dos modificadores (exemplo do Elfo). Nesse caso, seus pontos de vida serão iguais a 10.

3.3. Criação de um labirinto (10 pts)

O jogo deverá ser inicializado, apresentando três opções de dificuldade: fácil, normal e difícil. Na dificuldade fácil, o labirinto criado deverá caber em uma matriz 20×20. Já, na dificuldade média, o labirinto deverá estar contido em uma matriz 30×30. Por fim, na dificuldade máxima, o labirinto deverá ser contido em uma matriz 50×50.

A partir da dificuldade definida, uma breve narrativa envolvendo o personagem e por que o mesmo foi parar naquele labirinto deve ser apresentada ao jogador. Após a narrativa, o jogador irá ter seu personagem posicionado na entrada do labirinto.

Os labirintos serão matrizes bi-dimensionais e deverão existir, pelo menos, dois mapas para cada nível de dificuldade e este deverá ser sorteado no momento em que o jogador decidir o nível de dificuldade do jogo[1].

[1] Como sugestão, você pode colocar a escolha do nível na fase de criação do personagem (só será escolhida uma vez, por personagem) ou quando for escolhida a opção início do jogo (sorteado um mapa diferente toda vez que for iniciado)

Para cada mapa, crie uma matriz de inteiros com o tamanho desejado (20×20, 30×30 ou 50×50), onde o valor 0 representa espaços pelos quais o personagem poderá se mover livremente. Se o valor for 1, o espaço contém uma parede e o personagem não pode atravessar. E por fim, com o valor -1, que representará onde o jogador se encontra[2]. Lembre-se de que é um labirinto, então a bordas do mapa deverão ter paredes.

3.3.1. Movimentação e visão do personagem (10 pts)

O jogo deverá acontecer em turnos, onde o jogador poderá fornecer uma entrada do teclado por turno. Isto é, o jogador poderá movimentar seu personagem através do labirinto utilizando as teclas de movimento para a esquerda, movimento para baixo, movimento para a direita e movimento para cima. Para isso, pode-se configurar o teclado com as teclas A, S, D e W, que já aparecem comumente em outro jogos do gênero. Além disso, o jogador poderá utilizar teclas para aceitar opções e voltar ao menu principal do jogo. Para melhorar a fluidez do seu jogo, está sendo fornecido um arquivo objeto com funções auxiliares (`funcoes_auxiliares.h` e `funcoes_auxiliares.o`), o qual possui a função `le_tecla()` que é uma rotina de código para capturar teclas do usuário sem a necessidade de apertar a tecla `<enter>`. Abaixo, segue um exemplo de código com a sua utilização:

```
/* Instruções para compilar:
 * gcc -c teste.c -ansi
 * gcc -o programa teste.o funcoes_auxiliares.o -ansi
 */

#include <stdio.h>
#include "funcoes_auxiliares.h"

int main()
{
    int i;
    for (i = 0; i < 10; i++) {
        printf("%d\n", le_tecla());
    }
    return 0;
}
```

O labirinto é escuro e o personagem se movimenta utilizando uma tocha. Portanto, o mesmo só consegue enxergar parte do labirinto. Neste caso, limite o raio de visão do personagem a um raio de dois quadrados, conforme exemplificado na Figura 1. Utilize o comando `system("clear")` da biblioteca `stdlib.h` para limpar a tela antes de imprimir novamente o labirinto.

3.4. Criação de Eventos

Durante a criação do jogo principal, haverá sub-desafios. Esses desafios se resumirão a armadilhas, combates com monstros que estarão distribuídos aleatoriamente dentro do labirinto. Inclusive, haverá o embate com o chefe do labirinto.

3.4.1. Armadilhas (10 pts)

O labirinto possui armadilhas que estão espalhadas em seu interior. Aventureiros despercebidos sofrerão danos de 5 pontos toda vez que passarem sobre essas armadilhas. Uma armadilha só aparece no mapa caso

[2]Esses valores são os utilizados pelos desenvolvedores do jogo no código do jogo, e não necessariamente precisa ser o que aparece na tela para o jogdaor

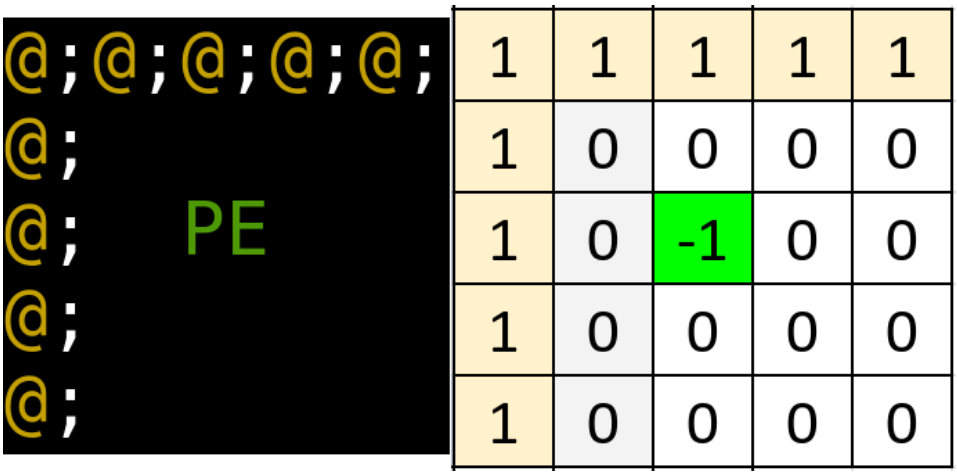

Figure 1. Exemplo de Visão do Logador do Labirinto.

esteja dentro do campo de visão do jogador e a mesma tenha sido percebida. O jogador terá a probabilidade de encontrar uma armadilha equivalente a sua $\frac{destreza+inteligencia}{20} * 100$, na primeira vez que a armadilha entrar em seu campo de visão. Caso haja uma falha, a armadilha não será percebida (não aparecerá para o jogador). Além das teclas de movimentação, deverá ter uma tecla chamada descoberta (tecla 'T', por exemplo), a qual o jogador não realiza um movimento, porém um teste de percepção deverá ser realizado, consumindo um ponto de vida o seu turno. Acerca da percepção dos personagens para armadilhas, no caso do Ladino, sua aguçada percepção permite que encontre armadilhas mais facilmente, logo o cálculo deverá ser $\frac{destreza+inteligencia}{12} * 100$.

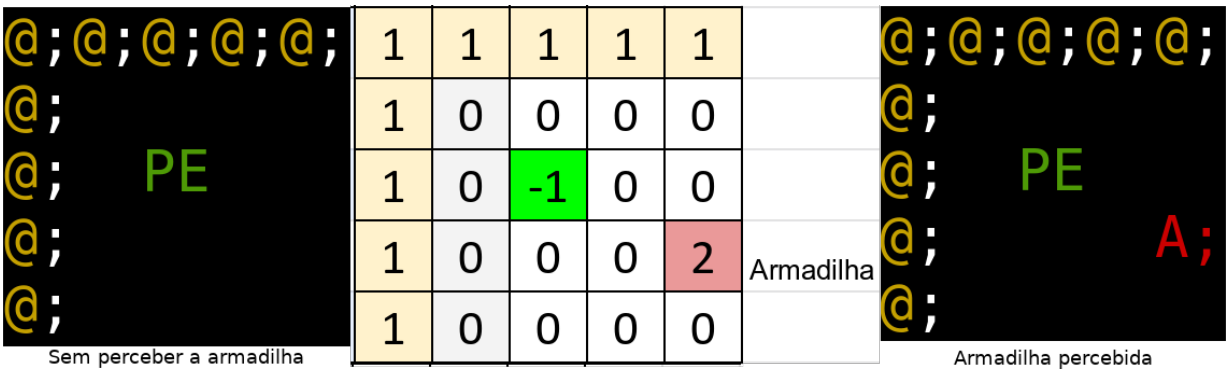

Figure 2. Exemplo de visão do jogador em relação a uma armadilha

Exemplo de jogada: o personagem Link é um guerreiro de inteligência 1 e destreza 2 e está se movendo pelo labirinto. Ao se mover, seu campo de visão cobre uma posição onde a armadilha (valor 2 no labirinto) está. No primeiro momento, ele jogou a probabilidade de percebe-la $\frac{1+2}{20} * 100 = 15\%$ de chance de descobri-la e falha. O jogador testa novamente a chance de Link encontrar uma armadilha, no caso, pressiona o botão de confirmação do teclado. Testa-se novamente os 15% e dessa vez, para surpresa de Link, existia uma armadilha no mapa. Sabendo da armadilha, Link poderá evita-la sem passar sobre ela e tomar o dano de 5 de vida.

3.4.2. Itens (10 pts)

Escondidos no labirinto, existem três tipos de poções, (*i*) Cura de Pontos de Vida, (*ii*) Potencializador de Atributo Principal e (*iii*) Visão Além do Alcance. O primeiro item irá curar um total de pontos de vida do personagem igual à sua constituição. Já, o segundo item, irá potencializar temporariamente durante x turnos (x é definido pelo desenvolvedor) os pontos de atributo principal (Força-Guerreiro, Inteligência-Mago e Destreza-Ladino) do personagem dobrando-os. Por fim, o terceiro item irá aumentar o campo de visão do jogador, de 2×2 para 4×4, de maneira temporária ou permanente. A quantidade e o posicionamento dos itens fica a critério do desenvolvedor. É interessante que os itens sejam proporcionais ao nível de dificuldade do jogo. Além disso, considere a representação dos itens na matriz de jogo, com o valor 3 - poção de cura, 4 - potencializador de atributos e 5 - pega a visão. Por fim, a forma da representação visual ao jogador ficará a critério do desenvolvedor.

3.4.3. Monstros (10 pts)

Dentro de um labirinto, existem vários monstros grotescos e famintos, em posições aleatórios, desejando carne nova para se alimentar. Esses monstros possuem valores únicos de ataque e defesa (valores arbitrários, escolhidos pelo desenvolvedor). Os posicionamentos dos monstros deverão ser aleatoriamente sorteados, mas com um raio de dez espaços distantes do personagem do jogador (para que ao início do jogo, não existam monstros no seu campo de visão).

Toda vez que um monstro encontra um jogador (o monstro está exatamente em um espaço adjacente ao jogador), ele realiza um ataque, que é dado por $ataque - forca$.

O jogador pode atacar o monstro, quando este estiver em uma posição adjacente, e esses ataques serão dados por uma tecla de confirmação/ação (B, por exemplo).

Após o ataque do seu personagem, o dano causado é de $1 + atributo_principal - defesa + acrescimos$. No caso de guerreiros, o dano é acrescido pela força do personagem. Por exemplo, Link acerta o monstro, logo, causa dano de $1 + 4 - 2 + 2 = 5$, que serão deduzidos dos pontos de vida do monstro. Ataques são feitos em área, então se houver mais de um monstro em volta do personagem, ele deve/pode atacar em todas as direções.

No caso de magos, suas explosões de gelo permitem ataques a distância, então é possível atacar a duas posições de distância do monstro.

O dano dos ataques dos monstros são dados por $ataque - forca + acrescimos$. Um exemplo de acréscimo é o do Ladino, que por ser ágil, reduz o dano do monstro em 2 pontos.

3.4.4. Chefe do Labirinto (10 pts)

Encontrar o chefe do labirinto é o objetivo principal do jogo. Somente derrotando o chefe do labirinto é que o jogo pode ser encerrado com a condição de vitória. O chefe do labirinto deverá ser posicionado em um local fixo do mapa. Fica a cargo do desenvolvedor decidir onde o chefe do labirinto deverá permanecer. Por exemplo, ele é aleatoriamente posicionado no extremo oposto do jogador no início do game. Ou, é uma

posição fixa dentro do mapa criado.

O chefe do labirinto quando encontrado, diferentemente dos outros monstros, ele apresentará dois desafios em sequência. Isto é, um jokenpô, seguido de uma competição com três charadas. Re-utilize o código do Trabalho 1. A cada vez que o jogador perder em um desses games, ele perderá pontos de vida, conforme definição a seguir.

Jokenpô - O jogador poderá jogar jokenpô com o computador e disputar uma melhor de três. Caso o jogador perca, ele perderá um ponto de vida. O jogo de jokenpô deverá ser implementado em uma função (sub-programa) utilizando uma de três opções (1-pedra, 2- papel e 3-tesoura). A jogada do computador deverá ser executada através de um número pseudo-aleatório entre 1 e 3.

Três Charadas - O jogador deverá acertar a resposta de três charadas. Para cada charada, o jogador deve receber um pequeno texto contendo três opções (1, 2 e 3). Das três opções, apenas uma será a resposta da charada. Cada vez que o jogador errar seu personagem perderá um ponto de vida. As charadas deverão ser implementadas em uma função (sub-programa) e aleatoriamente sorteadas a partir de um conjunto de dez charadas. Além disso, as 3 charadas apresentadas ao jogador não poderão ser repetidas entre si.

3.5. Condições de Término da Partida (Game Over) (5 pts)

O jogo deverá acabar quando o personagem derrotar o chefão ou chegar a zero pontos de vida e, assim, retornar para o menu principal apropriadamente.

3.6. Organização e Documentação (10 pts)

Documente todo o código (comentários) de maneira apropriada. Na elaboração de códigos, organize-o em diferentes arquivos, para uma melhor compreensão do mesmo. Exemplo, todas as declarações de funções, estruturas e variáveis globais em um arquivo de extensão `.h`, e as definições de funções em um ou mais arquivos `.c` a parte do arquivo que contém a função `main()` do seu programa.

3.7. Bônus

Os itens a seguir, são sugestões de bônus para o trabalho e não "precisam" ser feitas porém, caso alguma das mesmas sejam implementadas, serão avaliadas como bonificação para nota final deste trabalho.

3.7.1. Movimentação aleatória dos monstros (Bônus - 5 pts)

Na implementação solicitada, os monstros não se movimentam pelo labirinto, apenas ficam estacionados aguardando um evento de embate. Para deixar o jogo mais interessante, há a possibilidade de acrescentar uma movimentação aleatória aos monstros. Então, logo após a ação escolhida pelo jogador (movimentar, perceber ou atacar), os monstros também podem escolher, além de atacar, movimentar-se aleatoriamente (ou não) em uma de suas posições adjacentes ao mapa.

3.7.2. Salvar e Carregar o jogo (Bônus - 10 pts)

O jogo deverá poder ser salvo e carregado a qualquer momento. Para isso, o jogador deverá apertar a tecla de menu (M, por exemplo) e acessar a opção de salvar. O jogo poderá ser salvo inúmeras vezes,

sempre gerando ou sobrescrevendo um arquivo com extensão ".save". É importante que o jogo guarde o posicionamento do personagem, suas informações, seus atributos e pontos de vida. Além disso, os monstros também deverão ter seus posicionamentos, atributos e pontos de vida salvos. O mapa deverá ser salvo também, para que possa ser recuperado quando carregado.

3.7.3. Configuração de Teclas (Bônus - 10 pts)

O jogo também poderá fornecer algum meio de configurar as teclas do teclado a serem utilizadas. Por exemplo, crie um menu iterativo que peça ao jogador "**Entre com a tecla do comando 'para cima'**". O jogador digita a tecla que gostaria de configurar com esse comando, por exemplo, 'w'. Logo após, continue a configuração, por exemplo, "**Entre com a tecla do comando 'Ação ou Confirmação'**". E assim sucessivamente. Depois de configurar todas as teclas, as configurações deverão ser salvas e carregadas toda vez que o game for reiniciado.

Observação: Caso opte por não utilizar as teclas de configuração, o jogo deverá utilizar somente teclas comuns (caracteres ASCII), para evitar problemas de execução do jogo em computadores com diferentes *layouts* de teclados. Lembre-se de informar ao jogadores, em algum momento, sobre a mecânica do jogo.

Dica: Talvez seja melhor colocar essa opção logo no menu inicial do jogo.

4. Detalhes Acerca do Trabalho

4.1. Restrições deste Trabalho

Esta atividade pode ser feita em dupla.

4.2. Datas Importantes

Esse trabalho poderá ser submetido a partir de 05/06/2023 (segunda-feira) - 12:00h até 25/06/2023 (domingo) - 23:55h.

4.3. Por onde será a entrega o Trabalho?

O trabalho deverá ser entregue via Ambiente Virtual (https://ambientevirtual.idp.edu.br/), na disciplina de Algoritmos de Lógica de Programação. Na disciplina haverá uma atividade chamada "Trabalho Prático 02 (T2)", para submissão do trabalho.

4.4. O que deverá ser entregue, referente ao trabalho?

Deverá ser entregue o código produzido pelos alunos, relacionado à resolução do trabalho.

4.5. Como entregar o trabalho?

A submissão das atividades deverá ser feita em um arquivo único comprimido do tipo zip (`Zip archive data, at least v1.0 to extract`), contendo um diretório com o código elaborado. Além disso, para garantir a integridade do conteúdo entregue, o nome do arquivo comprimido deverá possuir duas informações (além da extensão `.zip`):

- As matrículas dos alunos, separada por hífen.
- O *hash* md5 do arquivo e `.zip`.

  Exemplo: `23001100-23001101-3b1b4a8e9a29cd6f91417d044e67faf2.zip`

Para gerar o md5 do arquivo comprimido, utilize o comando `md5sum` do Linux e em seguida faça o renomeamento utilizando o *hash* coletado.

Segue um exemplo de geração do arquivo final a ser submetido no moodle, para um aluno:

```
[13930] j3r3mias@tardis:trabalho-01 > ls
trabalho-01-apc-jeremias.c
[13931] j3r3mias@tardis:trabalho-01 > zip -r aaa.zip trabalho-01-apc-jeremias.c
  adding: trabalho-01-apc-jeremias.c (deflated 99%)
[13932] j3r3mias@tardis:trabalho-01 > ls
aaa.zip  trabalho-01-apc-jeremias.c
[13933] j3r3mias@tardis:trabalho-01 > ls -lha aaa.zip
-rw-rw-r-- 1 j3r3mias j3r3mias 335 out 15 16:54 aaa.zip
[13934] j3r3mias@tardis:trabalho-01 > md5sum aaa.zip
44cf46f9237cb506ebde3e9e1cd044f9  aaa.zip
[13935] j3r3mias@tardis:trabalho-01 > mv aaa.zip 160068000-44cf46f9237cb506ebde3e9e1cd044f9.zip
[13936] j3r3mias@tardis:trabalho-01 > ls -lha 160068000-44cf46f9237cb506ebde3e9e1cd044f9.zip
-rw-rw-r-- 1 j3r3mias j3r3mias 335 out 15 16:54 160068000-44cf46f9237cb506ebde3e9e1cd044f9.zip
[13937] j3r3mias@tardis:trabalho-01 > md5sum 160068000-44cf46f9237cb506ebde3e9e1cd044f9.zip
44cf46f9237cb506ebde3e9e1cd044f9  160068000-44cf46f9237cb506ebde3e9e1cd044f9.zip
[13938] j3r3mias@tardis:trabalho-01 >
```

4.6. Quem deverá entregar o trabalho?

Apenas um dos alunos deverá submeter o trabalho no Ambiente Virtual. Para isso, **somente um aluno irá gerar a versão final do trabalho (arquivo zip)**, para que o *hash* do arquivo não corra o risco de ficar diferente.

4.7. Observações importantes

- Não deixe para fazer o trabalho de última hora.
- Cópias de trabalho receberão zero (além disso, plágio é crime).
- Não deixe para fazer o trabalho de última hora.
- Não serão aceitos trabalhos com atraso.
- Não deixe para fazer o trabalho de última hora.
- Programas devrão ser feitos utilizando a linguagem C ANSI.
- **Não deixe para fazer o trabalho de última hora**.

5. Bibliografia

1. Cormen, T. et al., Algoritmos: Teoria e Prática. 3a ed., Elsevier - Campus, Rio de Janeiro, 2012
2. Ziviani, N., Projeto de Algoritmos com implementação em Pascal e C, 3a ed., Cengage Learning, 2010.
3. Evans, D., Introduction to Computing: explorations in Language, Logic, and Machines, CreatSpace, 2011.
4. Marcos A, Furlan et al., Algoritmos e Lógica de Programação., Cengage, 2011.
5. Kernighan, Brian W; Ritchie, Dennis M., C - a linguagem de programacao: Padrao ansi. Ed. Campus, 1989.
6. Farrer, Harry et al., Algoritmos estruturados. 3a ed., Editora LTC, 2011.